

CP Notebook: Tricky Implementation Patterns

Python-style pseudocode for the algorithms I tend to forget mid-solve.

DP Sanity Checklist

Before coding, write:

```
# state = ?
# answer = ?
# base = ?
# transition = ?
# order = ?
```

Usual bugs: missing empty state (0, None, empty interval), memo check too late, wrong loop order, giant state causing MLE, `mask == full` vs `mask == n`.

Tree DP: Return All States

Use one postorder pass when parent needs multiple answers.

```
def dfs(u):
    if not u:
        return (0, 0) # skip, take
    l_skip, l_take = dfs(u.left)
    r_skip, r_take = dfs(u.right)
    take = u.val + l_skip + r_skip
    skip = max(l_skip, l_take) + max(r_skip, r_take)
    return (skip, take)
```

Rule: avoid separate `take(u)` / `skip(u)` recursion unless memoized.

LCA

Single query on binary tree:

```
def lca(u, p, q):
    if not u or u == p or u == q:
        return u
    L = lca(u.left, p, q)
    R = lca(u.right, p, q)
    return u if L and R else L or R
```

Many tree queries: binary lifting.

```
up[0][v] = parent[v]
up[k][v] = up[k-1][up[k-1][v]]
```

Lift deeper node first, then jump both from high power down.

Interval DP

Contiguous segment, choose split/pivot.

```
for length in range(2, n + 1):
    for l in range(n - length + 1):
        r = l + length
        for m in range(l + 1, r):
            dp[l][r] = max(dp[l][r], dp[l][m] + dp[m][r] + cost(l, m, r))
```

Burst Balloons: add sentinel 1s. `dp[l][r]` = best inside open interval `(l, r)`. Pivot `m` is last popped.

Digit DP

State: pos, tight, started, extra_state.

```
from functools import cache
```

```
@cache
def dfs(pos, tight, started, state):
    if pos == len(digits):
        return valid(started, state)

    limit = digits[pos] if tight else 9
    ans = 0
    for d in range(limit + 1):
        ntight = tight and d == limit
        nstarted = started or d != 0
        ans += dfs(pos + 1, ntight, nstarted, trans(state, d, nstarted))
    return ans
```

If hand-memoizing, usually only memo when `tight == False`.

Bitmask DP

```
from functools import cache
```

```
full = (1 << n) - 1
```

```
@cache
def dfs(mask):
    if mask == full:
        return 0
    ans = INF
    for i in range(n):
        if not (mask & (1 << i)):
            ans = min(ans, cost(i, mask) + dfs(mask | (1 << i)))
    return ans
```

Checklist: `|` adds bit, `&` tests bit, base case reachable, memo active, loop indentation correct.

Shortest Paths

BFS: unweighted graph.

Dijkstra: nonnegative weights.

```
import heapq
```

```
dist = [INF] * n
dist[s] = 0
pq = [(0, s)]
```

```
while pq:
    d, u = heapq.heappop(pq)
    if d != dist[u]:
        continue
    for v, w in g[u]:
        if dist[v] > d + w:
            dist[v] = d + w
            heapq.heappush(pq, (dist[v], v))
```

Bellman-Ford: negative edges, negative cycle detection.

```
dist = [INF] * n
```

```

dist[s] = 0

for _ in range(n - 1):
    for u, v, w in edges:
        if dist[u] < INF and dist[v] > dist[u] + w:
            dist[v] = dist[u] + w

neg_cycle = False
for u, v, w in edges:
    if dist[u] < INF and dist[v] > dist[u] + w:
        neg_cycle = True

```

Why $n-1$: shortest simple path has at most $n-1$ edges. One more improvement means reachable negative cycle.

Minimum Spanning Tree

Kruskal: sort edges + DSU. Better when edge list is natural/sparse.

```

parent = list(range(n))
size = [1] * n

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    if size[ra] < size[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    size[ra] += size[rb]
    return True

mst = 0
for w, u, v in sorted(edges):
    if union(u, v):
        mst += w

```

Prim: grow from a node using cheapest crossing edge. Nice with adjacency list.

```

import heapq

seen = [False] * n
pq = [(0, 0)] # cost, node
mst = 0

while pq:
    w, u = heapq.heappop(pq)
    if seen[u]:
        continue
    seen[u] = True
    mst += w
    for v, cost in g[u]:
        if not seen[v]:
            heapq.heappush(pq, (cost, v))

```

If graph may be disconnected, count visited nodes or run Prim per component.

Toposort + DAG DP

```

from collections import deque

q = deque(i for i in range(n) if indeg[i] == 0)
order = []

while q:
    u = q.popleft()
    order.append(u)
    for v in g[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)

for u in order:
    for v, w in g[u]:
        dp[v] = max(dp[v], dp[u] + w)

If len(order) < n, graph has a cycle.

```

Strongly Connected Components

Kosaraju:

```

def dfs1(u):
    seen[u] = True
    for v in g[u]:
        if not seen[v]:
            dfs1(v)
    order.append(u)

def dfs2(u, c):
    comp[u] = c
    for v in rg[u]:
        if comp[v] == -1:
            dfs2(v, c)

seen = [False] * n
order = []
for i in range(n):
    if not seen[i]:
        dfs1(i)

comp = [-1] * n
c = 0
for u in reversed(order):
    if comp[u] == -1:
        dfs2(u, c)
        c += 1

```

SCC compression: build DAG with edges where `comp[u] != comp[v]`.

Max Flow

Ford-Fulkerson idea: keep finding augmenting paths in residual graph. BFS version = Edmonds-Karp.

```

from collections import deque

def bfs(s, t):
    parent = [-1] * n
    parent[s] = s
    q = deque([s])
    while q:
        u = q.popleft()

```

```

        for v in range(n):
            if parent[v] == -1 and cap[u][v] > 0:
                parent[v] = u
                if v == t:
                    return parent
                q.append(v)
    return None

flow = 0
while True:
    parent = bfs(s, t)
    if not parent:
        break
    push = INF
    v = t
    while v != s:
        u = parent[v]
        push = min(push, cap[u][v])
        v = u
    v = t
    while v != s:
        u = parent[v]
        cap[u][v] -= push
        cap[v][u] += push
        v = u
    flow += push

```

Reverse edges let you undo earlier flow choices.

Tree Diameter

Two BFS/DFS method for unweighted tree:

```

a, _ = farthest(0)
b, diameter = farthest(a)

```

Tree DP formula:

```

height[u] = 1 + max(height[child])
diameter = max(diameter, best_child_height_1 + best_child_height_2)

```

Be consistent: heights in edges gives diameter in edges; heights in nodes gives diameter in nodes.

Range Query Trees

Fenwick: prefix sums / frequencies.

```

bit = [0] * (n + 1)

```

```

def add(i, x):
    i += 1
    while i <= n:
        bit[i] += x
        i += i & -i

```

```

def sum_prefix(i):
    i += 1
    s = 0
    while i > 0:
        s += bit[i]
        i -= i & -i
    return s

```

```
def range_sum(l, r):
    return sum_prefix(r) - (sum_prefix(l - 1) if l else 0)
```

Segment tree: sum/min/max/gcd with point updates.

```
size = 1
while size < n:
    size *= 2
seg = [0] * (2 * size)

def set_val(i, x):
    i += size
    seg[i] = x
    i //= 2
    while i:
        seg[i] = seg[2*i] + seg[2*i+1]
        i //= 2

def query(l, r):
    # inclusive
    l += size
    r += size
    ans = 0
    while l <= r:
        if l % 2 == 1:
            ans += seg[l]
            l += 1
        if r % 2 == 0:
            ans += seg[r]
            r -= 1
        l //= 2
        r //= 2
    return ans
```

Use lazy propagation only for range updates.

Trie

Prefix tree for string sets / prefix queries.

```
class Trie:
    def __init__(self):
        self.children = {} # char -> child Trie node
        self.end = False # True if some inserted word ends exactly at this node

    def insert(self, word):
        node = self
        for c in word:
            # descend into child c, creating it on the fly if it doesn't exist yet
            node = node.children.setdefault(c, Trie())
        node.end = True # mark the last node reached as a real word end

    def _walk(self, word):
        # follow `word` from the root; returns the landing node, or None if
        # the path breaks partway (some prefix of `word` was never inserted)
        node = self
        for c in word:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

    def search(self, word):
        # exact word match: must reach the node AND it must be a word end,
        # not just a prefix of some longer inserted word
```

```

node = self._walk(word)
return node is not None and node.end

def starts_with(self, prefix):
    # prefix match only: reaching the node is enough, end doesn't matter
    return self._walk(prefix) is not None

```

Every op is $O(\text{len}(\text{word}))$ regardless of how many words are stored — cost is the string length, not the dictionary size.

Array children (`[None] * 26, index ord(c) - ord('a')`) is faster when the alphabet is small and fixed.

Binary trie for max XOR: insert numbers as root-to-leaf bit paths (MSB first), then for each query greedily prefer the branch with the opposite bit — that maximizes the XOR at that position, and greedy is safe since a high bit outweighs all lower bits combined.

```

BITS = 30    # highest bit index to consider; cover the max value in your input

```

```

def insert(root, x):
    node = root
    for b in range(BITS, -1, -1):          # MSB -> LSB, so numbers sharing a
        bit = (x >> b) & 1                # prefix share the same trie nodes
        if bit not in node:
            node[bit] = {}
        node = node[bit]

def max_xor(root, x):
    # walk the trie trying, at every bit, to go the OPPOSITE way from x's bit
    # (opposite bits XOR to 1); falls back to the same-bit child if the
    # opposite branch doesn't exist for any number inserted so far
    node, best = root, 0
    for b in range(BITS, -1, -1):
        bit = (x >> b) & 1
        want = bit ^ 1
        if want in node:
            best |= 1 << b    # this bit differs -> contributes to the XOR
            node = node[want]
        else:
            node = node[bit]
    return best

```

Pattern Map

Smell	Pattern
choose/skip on tree	tree DP states
contiguous segment	interval DP
count numbers $\leq N$	digit DP
subsets/visited set	bitmask DP
negative edges	Bellman-Ford
min network connection cost	MST: Kruskal/Prim
strongly connected directed groups	SCC
max bipartite/resource flow	max flow
prefix sums/frequencies	Fenwick
range min/max/sum + updates	segment tree

prefix lookup / autocomplete	trie
max XOR pair	binary trie